



Network Management

Having made our way through the first eight chapters of this text, we're now well aware that a network consists of *many* complex, interacting pieces of hardware and software—from the links, switches, routers, hosts, and other devices that comprise the physical components of the network to the many protocols (in both hardware and software) that control and coordinate these devices. When hundreds or thousands of such components are cobbled together by an organization to form a network, it is not surprising that components will occasionally malfunction, that network elements will be misconfigured, that network resources will be overutilized, or that network components will simply “break” (for example, a cable will be cut or a can of soda will be spilled on top of a router). The network administrator, whose job it is to keep the network “up and running,” must be able to respond to (and better yet, avoid) such mishaps. With potentially thousands of network components spread out over a wide area, the network administrator in a network operations center (NOC) clearly needs tools to help monitor, manage, and control the network. In this chapter, we'll examine the architecture, protocols, and information base used by a network administrator in this task.

9.1 What Is Network Management?

Before diving in to network management itself, let's first consider a few illustrative “real-world” non-networking scenarios in which a complex system with many interacting components must be monitored, managed, and controlled by an administrator. Electrical power-generation plants have a control room where dials, gauges, and lights monitor the status (temperature, pressure, flow) of remote valves, pipes, vessels, and other plant components. These devices allow the operator to monitor the plant's many components, and may alert the operator (with the famous flashing red warning light) when trouble is imminent. Actions are taken by the plant operator to control these components. Similarly, an airplane cockpit is instrumented to allow a pilot to monitor and control the many components that make up an airplane. In these two examples, the “administrator” *monitors* remote devices and *analyzes* their data to ensure that they are operational and operating within prescribed limits (for example, that a core meltdown of a nuclear power plant is not imminent, or that the plane is not about to run out of fuel), *reactively controls* the system by making adjustments in response to the changes within the system or its environment, and *proactively manages* the system (for example, by detecting trends or anomalous behavior, allowing action to be taken before serious problems arise). In a similar sense, the network administrator will actively monitor, manage, and control the system with which she or he is entrusted.

In the early days of networking, when computer networks were research artifacts rather than a critical infrastructure used by hundreds of millions of people a day, “network management” was unheard of. If one encountered a network problem, one might run a few pings to locate the source of the problem and then modify system settings, reboot hardware or software, or call a remote colleague to do so. (A very readable discussion of the first major “crash” of the ARPAnet on October 27, 1980, long before network management tools were available, and the efforts taken to recover from and understand the crash is [RFC 789].) As the public Internet and private intranets have grown from small networks into a large global infrastructure, the need to manage the huge number of hardware and software components within these networks more systematically has grown more important as well.

In order to motivate our study of network management, let's begin with a simple example. Figure 9.1 illustrates a small network consisting of three routers and a number of hosts and servers. Even in such a simple network, there are many scenarios in which a network administrator might benefit tremendously from having appropriate network management tools:

- *Detecting failure of an interface card at a host or a router.* With appropriate network management tools, a network entity (for example, router A) may report to the network administrator that one of its interfaces has gone down. (This is certainly preferable to a phone call to the NOC from an irate user who says the network connection is down!) A network administrator who actively monitors

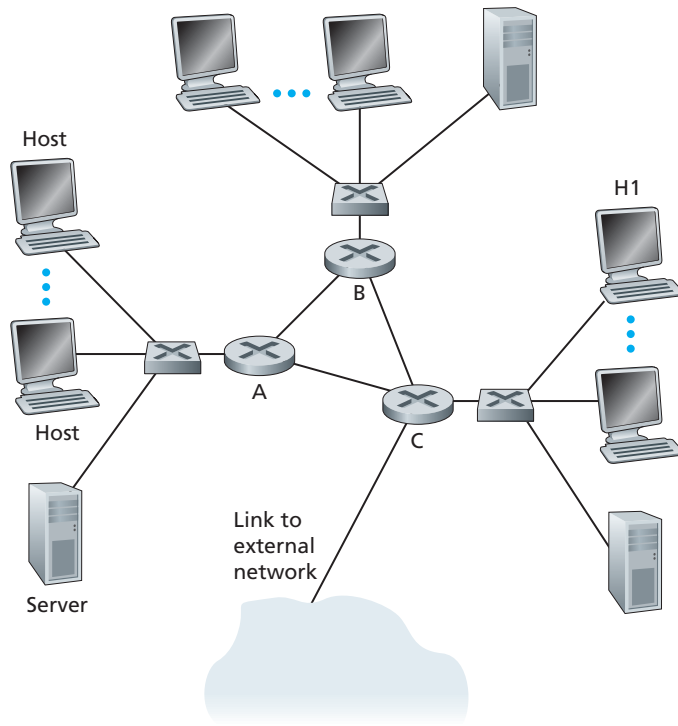


Figure 9.1 ♦ A simple scenario illustrating the uses of network management

and analyzes network traffic may be able to *really* impress the would-be irate user by detecting problems in the interface ahead of time and replacing the interface card before it fails. This might be done, for example, if the administrator noted an increase in checksum errors in frames being sent by the soon-to-die interface.

- *Host monitoring.* Here, the network administrator might periodically check to see if all network hosts are up and operational. Once again, the network administrator may really be able to impress a network user by proactively responding to a problem (host down) before it is reported by a user.
- *Monitoring traffic to aid in resource deployment.* A network administrator might monitor source-to-destination traffic patterns and notice, for example, that by switching servers between LAN segments, the amount of traffic that crosses multiple LANs could be significantly decreased. Imagine the happiness all around when better performance is achieved with no new equipment costs. Similarly, by monitoring link utilization, a network administrator might determine that a LAN segment or the external link to the outside world is overloaded and

that a higher-bandwidth link should thus be provisioned (alas, at an increased cost). The network administrator might also want to be notified automatically when congestion levels on a link exceed a given threshold value, in order to provision a higher-bandwidth link before congestion becomes serious.

- *Detecting rapid changes in routing tables.* Route flapping—frequent changes in the routing tables—may indicate instabilities in the routing or a misconfigured router. Certainly, the network administrator who has improperly configured a router would prefer to discover the error him- or herself, before the network goes down.
- *Monitoring for SLAs.* **Service Level Agreements (SLAs)** are contracts that define specific performance metrics and acceptable levels of network-provider performance with respect to these metrics [Huston 1999a]. Verizon and Sprint are just two of the many network providers that guarantee SLAs [AT&T SLA 2012; Verizon SLA 2012] to their customers. These SLAs include service availability (outage), latency, throughput, and outage notification requirements. Clearly, if performance criteria are to be part of a service agreement between a network provider and its users, then measuring and managing performance will be of great importance to the network administrator.
- *Intrusion detection.* A network administrator may want to be notified when network traffic arrives from, or is destined for, a suspicious source (for example, host or port number). Similarly, a network administrator may want to detect (and in many cases filter) the existence of certain types of traffic (for example, source-routed packets, or a large number of SYN packets directed to a given host) that are known to be characteristic of the types of security attacks that we considered in Chapter 8.

The International Organization for Standardization (ISO) has created a network management model that is useful for placing the anecdotal scenarios above in a more structured framework. Five areas of network management are defined:

- *Performance management.* The goal of performance management is to quantify, measure, report, analyze, and control the performance (for example, utilization and throughput) of different network components. These components include individual devices (for example, links, routers, and hosts) as well as end-to-end abstractions such as a path through the network. We will see shortly that protocol standards such as the Simple Network Management Protocol (SNMP) [RFC 3410] play a central role in Internet performance management.
- *Fault management.* The goal of fault management is to log, detect, and respond to fault conditions in the network. The line between fault management and performance management is rather blurred. We can think of fault management as the immediate handling of transient network failures (for example, link, host, or router hardware or software outages), while performance management takes

the longer-term view of providing acceptable levels of performance in the face of varying traffic demands and occasional network device failures. As with performance management, the SNMP protocol plays a central role in fault management.

- *Configuration management.* Configuration management allows a network manager to track which devices are on the managed network and the hardware and software configurations of these devices. An overview of configuration management and requirements for IP-based networks can be found in [RFC 3139].
- *Accounting management.* Accounting management allows the network manager to specify, log, and control user and device access to network resources. Usage quotas, usage-based charging, and the allocation of resource-access privileges all fall under accounting management.
- *Security management.* The goal of security management is to control access to network resources according to some well-defined policy. The key distribution centers that we studied in Section 8.3 are components of security management. The use of firewalls to monitor and control external access points to one's network, a topic we studied in Section 8.9, is another crucial component.

In this chapter, we'll cover only the rudiments of network management. Our focus will be purposefully narrow—we'll examine only the *infrastructure* for network management—the overall architecture, network management protocols, and information base through which a network administrator keeps the network up and running. We'll *not* cover the decision-making processes of the network administrator, who must plan, analyze, and respond to the management information that is conveyed to the NOC. In this area, topics such as fault identification and management [Katzela 1995; Medhi 1997; Labovitz 1997; Steinder 2002; Feamster 2005; Wu 2005; Teixeira 2006], anomaly detection [Lakhina 2004; Lakhina 2005; Barford 2009], and more come into consideration. Nor will we cover the broader topic of service management [Saydam 1996; RFC 3052]—the provisioning of resources such as bandwidth, server capacity, and the other computational/communication resources needed to meet the mission-specific service requirements of an enterprise.

An often-asked question is “What is network management?” Our discussion above has motivated the need for, and illustrated a few of the uses of, network management. We'll conclude this section with a single-sentence (albeit a rather long run-on sentence) definition of network management from [Saydam 1996]:

“Network management includes the deployment, integration, and coordination of the hardware, software, and human elements to monitor, test, poll, configure, analyze, evaluate, and control the network and element resources to meet the real-time, operational performance, and Quality of Service requirements at a reasonable cost.”

It's a mouthful, but it's a good workable definition. In the following sections, we'll add some meat to this rather bare-bones definition of network management.

9.2 The Infrastructure for Network Management

We've seen in the preceding section that network management requires the ability to “monitor, test, poll, configure, . . . and control” the hardware and software components in a network. Because the network devices are distributed, this will, at a minimum, require that the network administrator be able to gather data (for example, for monitoring purposes) from a remote entity and effect changes at that remote entity (for example, control it). A human analogy will prove useful here for understanding the infrastructure needed for network management.

Imagine that you're the head of a large organization that has branch offices around the world. It's your job to make sure that the pieces of your organization are operating smoothly. How will you do so? At a minimum, you'll periodically gather data from your branch offices in the form of reports and various quantitative measures of activity, productivity, and budget. You'll occasionally (but not always) be explicitly notified when there's a problem in one of the branch offices; the branch manager who wants to climb the corporate ladder (perhaps to get your job) may send you unsolicited reports indicating how smoothly things are running at his or her branch. You'll sift through the reports you receive, hoping to find smooth operations everywhere but no doubt finding problems in need of your attention. You might initiate a one-on-one dialogue with one of your problem branch offices, gather more data in order to understand the problem, and then pass down an executive order (“Make this change!”) to the branch office manager.

Implicit in this very common human scenario is an infrastructure for controlling the organization—the boss (you), the remote sites being controlled (the branch offices), your remote agents (the branch office managers), communication protocols (for transmitting standard reports and data, and for one-on-one dialogues), and data (the report contents and the quantitative measures of activity, productivity, and budget). Each of these components in human organizational management has a counterpart in network management.

The architecture of a network management system is conceptually identical to this simple human organizational analogy. The network management field has its own specific terminology for the various components of a network management architecture, and so we adopt that terminology here. As shown in Figure 9.2, there are three principal components of a network management architecture: a managing entity (the boss in our analogy above—you), the managed devices (the branch office), and a network management protocol.

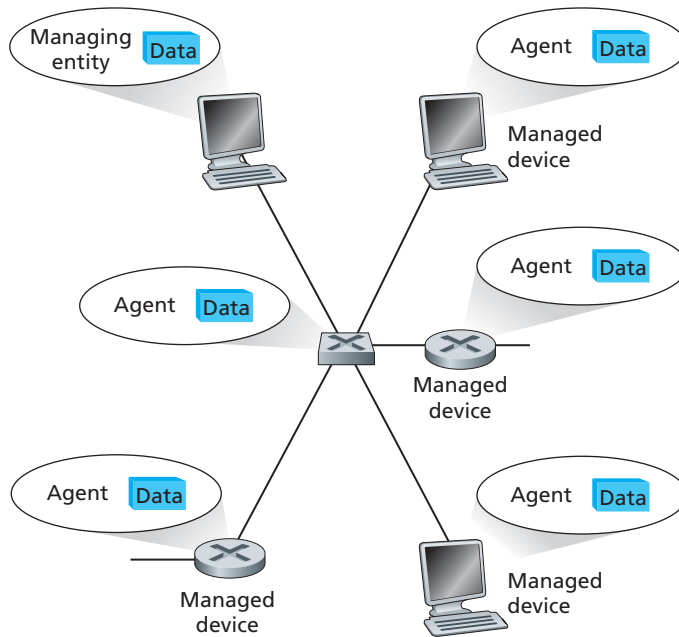


Figure 9.2 ♦ Principal components of a network management architecture

The **managing entity** is an application, typically with a human in the loop, running in a centralized network management station in the NOC. The managing entity is the locus of activity for network management; it controls the collection, processing, analysis, and/or display of network management information. It is here that actions are initiated to control network behavior and here that the human network administrator interacts with the network devices.

A **managed device** is a piece of network equipment (including its software) that resides on a managed network. This is the branch office in our human analogy. A managed device might be a host, router, bridge, hub, printer, or modem. Within a managed device, there may be several so-called **managed objects**. These managed objects are the actual pieces of hardware within the managed device (for example, a network interface card), and the sets of configuration parameters for the pieces of hardware and software (for example, an intradomain routing protocol such as RIP). In our human analogy, the managed objects might be the departments within the branch office. These managed objects have pieces of information associated with them that are collected into a **Management Information Base**

(MIB); we'll see that the values of these pieces of information are available to (and in many cases able to be set by) the managing entity. In our human analogy, the MIB corresponds to quantitative data (measures of activity, productivity, and budget, with the latter being setttable by the managing entity!) exchanged between the branch office and the main office. We'll study MIBs in detail in Section 9.3. Finally, also resident in each managed device is a **network management agent**, a process running in the managed device that communicates with the managing entity, taking local actions at the managed device under the command and control of the managing entity. The network management agent is the branch manager in our human analogy.

The third piece of a network management architecture is the **network management protocol**. The protocol runs between the managing entity and the managed devices, allowing the managing entity to query the status of managed devices and indirectly take actions at these devices via its agents. Agents can use the network management protocol to inform the managing entity of exceptional events (for example, component failures or violation of performance thresholds). It's important to note that the network management protocol does not itself manage the network. Instead, it provides capabilities that a network administrator can use to manage ("monitor, test, poll, configure, analyze, evaluate, and control") the network. This is a subtle, but important, distinction.

Although the infrastructure for network management is conceptually simple, one can often get bogged down with the network-management-speak vocabulary of "managing entity," "managed device," "managing agent," and "Management Information Base." For example, in network-management-speak, in our simple host-monitoring scenario, "managing agents" located at "managed devices" are periodically queried by the "managing entity"—a simple idea, but a linguistic mouthful! With any luck, keeping in mind the human organizational analogy and its obvious parallels with network management will be of help as we continue through this chapter.

Our discussion of network management architecture above has been generic, and broadly applies to a number of the network management standards and efforts that have been proposed over the years. Network management standards began maturing in the late 1980s, with OSI **CMISE/CMIP** (the **Common Management Information Services Element/Common Management Information Protocol**) [Piscatello 1993; Stallings 1993; Glitho 1998] and the Internet **SNMP** (**Simple Network Management Protocol**) [RFC 3410; Stallings 1999; Rose 1996] emerging as the two most important standards [Subramanian 2000]. Both are designed to be independent of vendor-specific products or networks. Because SNMP was quickly designed and deployed at a time when the need for network management was becoming painfully clear, SNMP found widespread use and acceptance. Today, SNMP has emerged as the most widely used and deployed network management framework. We'll cover SNMP in detail in the following section.



PRINCIPLES IN PRACTICE

COMCAST'S NETWORK OPERATIONS CENTER

Comcast's world-class fiber-based IP network delivers converged products and services to 49 million combined video, data and voice customers. Comcast's network includes more than 618,000 plant route miles, 138,000 fiber route miles, 30,000 backbone miles, 122,000 optical nodes, and massive storage for the Comcast Content Delivery Network, which delivers a Video on Demand product of more than 134 Terabytes. Each part of Comcast's network, up to and including the customers' homes or businesses, is monitored by one of the company's Operations Centers.

Comcast operates two National Network Operations Centers that manage the national backbone, regional area networks, national applications and specific platforms supporting voice, data and video infrastructure for residential, commercial and wholesale customers. In addition, Comcast has three Divisional Operations Centers that manage the local infrastructure that supports all of their customers. Both the National and Divisional Operations Centers are accountable for proactively monitoring all aspects of their network and product performance on a 7 x 24 x 365 basis, utilizing common processes and systems. For example, various network events at the national and local levels have common pre-defined severity levels, recovery processes, and expected Mean Time to Restore objectives. The national and divisional centers can back up each other if a local issue impacts a site's operation. In addition, the National and Divisional Operations Centers have an extensive Virtual Private Network that allows engineers to securely access the network to remotely perform proactive or reactive network management activities.

Comcast's approach to network management involves five key areas: Performance Management, Fault Management, Configuration Management, Accounting Management and Security Management. **Performance Management** is focused on understanding



These screens show tools supporting correlation, threshold management, ticketing used by Comcast technicians (Courtesy of Comcast.)

(continues)

how the network/systems and applications (collectively referred to as the ecosystem) are performing with respect to pre-defined measures specific to time of day, day of week, or special events (e.g., storm surges or pay events, such as a boxing match). These pre-defined performance measures exist throughout the service path, from the customer's residence or business through the entire network, as well as the interface points to partners and peers. In addition, synthetic transactions are run to ensure the health of the ecosystem on a continual basis. **Fault Management** is defined as the ability to detect, log and understand anomalies that may impact customers. Comcast utilizes correlation engines to properly determine an event's severity and act appropriately, eliminating or remediating potential issues before they affect customers. **Configuration Management** makes sure appropriate versions of hardware and software are in place across all elements of the ecosystem. Keeping these elements at their peak "golden" levels helps them avoid unintended consequences. **Accounting Management** ensures that the operations centers have a clear understanding of the provisioning and utilization of the ecosystem. This is especially important to ensure that at all times the operations centers have the ability to re-route traffic effectively. **Security Management** ensures that the proper controls exist to ensure the ecosystem is effectively protected against inappropriate access.

Network Operations Centers and the ecosystem they support are not static. Engineering and Operations personnel are constantly re-evaluating the pre-defined performance measures and tools to ensure that the customers' expectations for operational excellence are met.

9.3 The Internet-Standard Management Framework

Contrary to what the name SNMP (Simple Network Management Protocol) might suggest, network management in the Internet is much more than just a protocol for moving management data between a management entity and its agents, and has grown to be much more complex than the word "simple" might suggest. The current Internet-Standard Management Framework traces its roots back to the Simple Gateway Monitoring Protocol, SGMP [RFC 1028]. SGMP was designed by a group of university network researchers, users, and managers, whose experience with SGMP allowed them to design, implement, and deploy SNMP in just a few months [Lynch 1993]—a far cry from today's rather drawn-out standardization process. Since then, SNMP has evolved from SNMPv1 through SNMPv2 to the most recent version, SNMPv3 [RFC 3410], released in April 1999 and updated in December 2002.

When describing any framework for network management, certain questions must inevitably be addressed:

- What (from a semantic viewpoint) is being monitored? And what form of control can be exercised by the network administrator?
- What is the specific form of the information that will be reported and/or exchanged?
- What is the communication protocol for exchanging this information?

Recall our human organizational analogy from the previous section. The boss and the branch managers will need to agree on the measures of activity, productivity, and budget used to report the branch office's status. Similarly, they'll need to agree on the actions the boss can take (for example, cut the budget, order the branch manager to change some aspect of the office's operation, or fire the staff and shut down the branch office). At a lower level of detail, they'll need to agree on the form in which this data is reported. For example, in what currency (dollars, euros?) will the budget be reported? In what units will productivity be measured? While these may seem like trivial details, they must be agreed upon, nonetheless. Finally, the manner in which information is conveyed between the main office and the branch offices (that is, their communication protocol) must be specified.

The Internet-Standard Management Framework addresses the questions posed above. The framework consists of four parts:

- Definitions of *network management objects*, known as MIB objects. In the Internet-Standard Management Framework, management information is represented as a collection of managed objects that together form a virtual information store, known as the Management Information Base (MIB). An MIB object might be a counter, such as the number of IP datagrams discarded at a router due to errors in an IP datagram header, or the number of carrier sense errors in an Ethernet interface card; descriptive information such as the version of the software running on a DNS server; status information such as whether a particular device is functioning correctly; or protocol-specific information such as a routing path to a destination. MIB objects thus define the management information maintained by a managed device. Related MIB objects are gathered into **MIB modules**. In our human organizational analogy, the MIB defines the information conveyed between the branch office and the main office.
- A *data definition language*, known as SMI (Structure of Management Information). SMI defines the data types, an object model, and rules for writing and revising management information. MIB objects are specified in this data definition language. In our human organizational analogy, the SMI is used to define the details of the *format* of the information to be exchanged.
- A *protocol*, *SNMP*. SNMP is used for conveying information and commands between a managing entity and an agent executing on behalf of that entity within a managed network device.
- *Security and administration capabilities*. The addition of these capabilities represents the major enhancement in SNMPv3 over SNMPv2.

The Internet network management architecture is thus modular by design, with a protocol-independent data definition language and MIB, and an MIB-independent protocol. Interestingly, this modular architecture was first put in place to ease the transition from an SNMP-based network management to a network management framework being developed by ISO, the competing network management architecture when SNMP was first conceived—a transition that never occurred. Over time, however, SNMP’s design modularity has allowed it to evolve through three major revisions, with each of the four major parts of SNMP discussed above evolving independently. Clearly, the right decision about modularity was made, even if for the wrong reason!

In the following subsections, we cover the four major components of the Internet-Standard Management Framework in more detail.

9.3.1 Structure of Management Information: SMI

The **Structure of Management Information, SMI** (a rather oddly named component of the network management framework whose name gives no hint of its functionality), is the language used to define the management information residing in a managed-network entity. Such a definition language is needed to ensure that the syntax and semantics of the network management data are well defined and unambiguous. Note that the SMI does not define a specific instance of the data in a managed-network entity, but rather the language in which such information is specified. The documents describing the SMI for SNMPv3 (which rather confusingly, is called SMIv2) are [RFC 2578; RFC 2579; RFC 2580]. Let’s examine the SMI in a bottom-up manner, starting with the base data types in the SMI. We’ll then look at how managed objects are described in SMI, then how related managed objects are grouped into modules.

SMI Base Data Types

RFC 2578 specifies the basic data types in the SMI MIB module-definition language. Although the SMI is based on the ASN.1 (Abstract Syntax Notation One) [ISO X.680 2002] object-definition language (see Section 9.4), enough SMI-specific data types have been added that SMI should be considered a data definition language in its own right. The 11 basic data types defined in RFC 2578 are shown in Table 9.1. In addition to these scalar objects, it is also possible to impose a tabular structure on an ordered collection of MIB objects using the SEQUENCE OF construct; see RFC 2578 for details. Most of the data types in Table 9.1 will be familiar (or self-explanatory) to most readers. The one data type we will discuss in more detail shortly is the OBJECT IDENTIFIER data type, which is used to name an object.

SMI Higher-Level Constructs

In addition to the basic data types, the SMI data definition language also provides higher-level language constructs.

Data Type	Description
INTEGER	32-bit integer, as defined in ASN.1, with a value between -2^{31} and $2^{31} - 1$ inclusive, or a value from a list of possible named constant values.
Integer32	32-bit integer with a value between -2^{31} and $2^{31} - 1$ inclusive.
Unsigned32	Unsigned 32-bit integer in the range 0 to $2^{32} - 1$ inclusive.
OCTET STRING	ASN.1-format byte string representing arbitrary binary or textual data, up to 65,535 bytes long.
OBJECT IDENTIFIER	ASN.1-format administratively assigned (structured name); see Section 9.3.2.
IPAddress	32-bit Internet address, in network-byte order.
Counter32	32-bit counter that increases from 0 to $2^{32} - 1$ and then wraps around to 0.
Counter64	64-bit counter.
Gauge32	32-bit integer that will not count above $2^{32} - 1$ nor decrease beyond 0 when increased or decreased.
TimeTicks	Time, measured in 1/100ths of a second since some event.
Opaque	Uninterpreted ASN.1 string, needed for backward compatibility.

Table 9.1 ♦ Basic data types of the SMI

The OBJECT-TYPE construct is used to specify the data type, status, and semantics of a managed object. Collectively, these managed objects contain the management data that lies at the heart of network management. There are more than 10,000 defined objects in various Internet RFCs [RFC 3410]. The OBJECT-TYPE construct has four clauses. The SYNTAX clause of an OBJECT-TYPE definition specifies the basic data type associated with the object. The MAX-ACCESS clause specifies whether the managed object can be read, be written, be created, or have its value included in a notification. The STATUS clause indicates whether the object definition is current and valid, obsolete (in which case it should not be implemented, as its definition is included for historical purposes only), or deprecated (obsolete, but implementable for interoperability with older implementations). The DESCRIPTION clause contains a human-readable textual definition of the object; this “documents” the purpose of the managed object and should provide all the semantic information needed to implement the managed object.

As an example of the OBJECT-TYPE construct, consider the `ipSystemStatsInDelivers` object-type definition from [RFC 4293]. This object defines a 32-bit counter that keeps track of the number of IP datagrams that were received at the managed device and were successfully delivered to an upper-layer protocol.

The final line of this definition is concerned with the name of this object, a topic we'll consider in the following subsection.

```
ipSystemStatsInDelivers OBJECT-TYPE
    SYNTAX      Counter32
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "The total number of datagrams successfully
        delivered to IPuser-protocols (including ICMP).

        When tracking interface statistics, the counter
        of the interface to which these datagrams were
        addressed is incremented. This interface might
        not be the same as the input interface for
        some of the datagrams.

        Discontinuities in the value of this counter can
        occur at re-initialization of the management
        system, and at other times as indicated by the
        value of ipSystemStatsDiscontinuityTime."
 ::= { ipSystemStatsEntry 18 }
```

The MODULE-IDENTITY construct allows related objects to be grouped together within a “module.” For example, [RFC 4293] specifies the MIB module that defines managed objects (including `ipSystemStatsInDelivers`) for managing implementations of the Internet Protocol (IP) and its associated Internet Control Message Protocol (ICMP). [RFC 4022] specifies the MIB module for TCP, and [RFC 4113] specifies the MIB module for UDP. [RFC 4502] defines the MIB module for RMON remote monitoring. In addition to containing the OBJECT-TYPE definitions of the managed objects within the module, the MODULE-IDENTITY construct contains clauses to document contact information of the author of the module, the date of the last update, a revision history, and a textual description of the module. As an example, consider the module definition for management of the IP protocol:

```
ipMIB MODULE-IDENTITY
    LAST-UPDATED "200602020000Z"
    ORGANIZATION "IETF IPv6 MIB Revision Team"
    CONTACT-INFO
        "Editor:
        Shawn A. Routhier
        Interworking Labs
        108 Whispering Pines Dr. Suite 235
```

```

    Scotts Valley, CA 95066
    USA
    EMail: <sar@iwl.com>"
DESCRIPTION
    "The MIB module for managing IP and ICMP
    implementations, but excluding their
    management of IP routes.

    Copyright (C) The Internet Society (2006).
    This version of this MIB module is part of
    RFC 4293; see the RFC itself for full legal
    notices."

REVISION      "200602020000Z"
DESCRIPTION
    "The IP version neutral revision with added
    IPv6 objects for ND, default routers, and
    router advertisements. As well as being the
    successor to RFC 2011, this MIB is also the
    successor to RFCs 2465 and 2466. Published
    as RFC 4293."

REVISION      "199411010000Z"
DESCRIPTION
    "A separate MIB module (IP-MIB) for IP and
    ICMP management objects. Published as RFC
    2011."

REVISION      "199103310000Z"
DESCRIPTION
    "The initial revision of this MIB module was
    part of MIB-II, which was published as RFC
    1213."
::= { mib-2 48}

```

The NOTIFICATION-TYPE construct is used to specify information regarding SNMPv2-Trap and InformationRequest messages generated by an agent, or a managing entity; see Section 9.3.3. This information includes a textual DESCRIPTION of when such messages are to be sent, as well as a list of values to be included in the message generated; see [RFC 2578] for details. The MODULE-COMPLIANCE construct defines the set of managed objects within a module that an agent must implement. The AGENT-CAPABILITIES construct specifies the capabilities of agents with respect to object- and event-notification definitions.

9.3.2 Management Information Base: MIB

As noted previously, the **Management Information Base, MIB**, can be thought of as a virtual information store, holding managed objects whose values collectively reflect the current “state” of the network. These values may be queried and/or set by a managing entity by sending SNMP messages to the agent that is executing in a managed device on behalf of the managing entity. Managed objects are specified using the OBJECT-TYPE SMI construct discussed above and gathered into **MIB modules** using the MODULE-IDENTITY construct.

The IETF has been busy standardizing the MIB modules associated with routers, hosts, and other network equipment. This includes basic identification data about a particular piece of hardware, and management information about the device’s network interfaces and protocols. As of 2006 there were more than 200 standards-based MIB modules and an even larger number of vendor-specific (private) MIB modules. With all of these standards, the IETF needed a way to identify and name the standardized modules as well as the specific managed objects within a module. Rather than start from scratch, the IETF adopted a standardized object identification (naming) framework that had already been put in place by the International Organization for Standardization (ISO). As is the case with many standards bodies, the ISO had “grand plans” for its standardized object identification framework—to identify every possible standardized object (for example, data format, protocol, or piece of information) in any network, regardless of the network standards organization (for example, Internet IETF, ISO, IEEE, or ANSI), equipment manufacturer, or network owner. A lofty goal indeed! The object identification framework adopted by ISO is part of the ASN.1 (Abstract Syntax Notation One) [ISO X.680 2002] object definition language that we’ll discuss in Section 9.4. Standardized MIB modules have their own cozy corner in this all-encompassing naming framework, as discussed below.

As shown in Figure 9.3, objects are named in the ISO naming framework in a hierarchical manner. Note that each branch point in the tree has both a name and a number (shown in parentheses); any point in the tree is thus identifiable by the sequence of names or numbers that specify the path from the root to that point in the identifier tree. A fun, but incomplete and unofficial, Web-based utility for traversing part of the object identifier tree (using branch information contributed by volunteers) may be found in [OID Repository 2012].

At the top of the hierarchy are the ISO and the Telecommunication Standardization Sector of the International Telecommunication Union (ITU-T), the two main standards organizations dealing with ASN.1, as well as a branch for joint efforts by these two organizations. Under the ISO branch of the tree, we find entries for all ISO standards (1.0) and for standards issued by standards bodies of various ISO-member countries (1.2). Although not shown in Figure 9.3, under (ISO member body, a.k.a. 1.2) we would find USA (1.2.840), under which we would find a number of IEEE, ANSI, and company-specific standards. These include RSA (1.2.840.11359) and Microsoft (1.2.840.113556), under which we find the Microsoft File Formats (1.2.840.113556.4) for various Microsoft products, such as

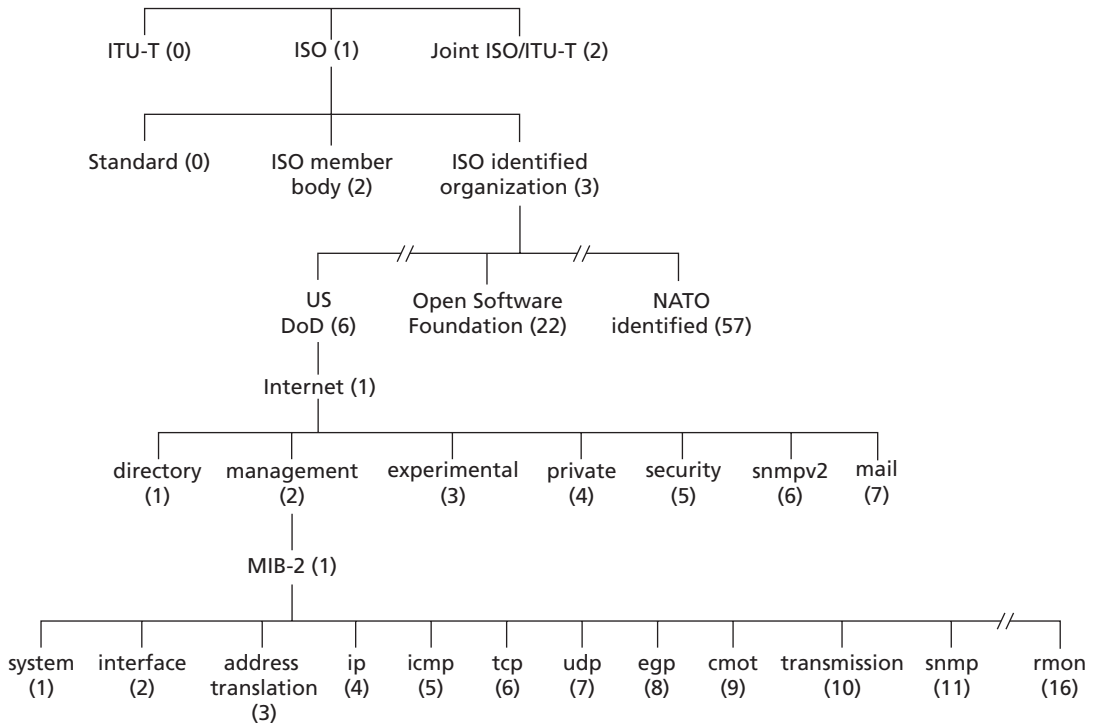


Figure 9.3 ♦ ASN.1 object identifier tree

Word (1.2.840.113556.4.2). But we are interested here in networking (*not* Microsoft Word files), so let us turn our attention to the branch labeled 1.3, the standards issued by bodies recognized by the ISO. These include the U.S. Department of Defense (6) (under which we will find the Internet standards), the Open Software Foundation (22), the airline association SITA (69), NATO-identified bodies (57), as well as many other organizations.

Under the **Internet** branch of the tree (1.3.6.1), there are seven categories. Under the **private** (1.3.6.1.4) branch, we find a list [IANA 2009b] of the names and private enterprise codes of many thousands of private companies that have registered with the Internet Assigned Numbers Authority (IANA) [IANA 2009a]. Under the **management** (1.3.6.1.2) and **MIB-2** branches (1.3.6.1.2.1) of the object identifier tree, we find the definitions of the standardized MIB modules. Whew—it's a long journey down to our corner of the ISO name space!

Standardized MIB Modules

The lowest level of the tree in Figure 9.3 shows some of the important hardware-oriented MIB modules (**system** and **interface**) as well as modules associated

with some of the most important Internet protocols. [RFC 5000] lists all of the standardized MIB modules as of 2008. While MIB-related RFCs make for rather tedious and dry reading, it is instructive (that is, like eating vegetables, it is “good for you”) to consider a few MIB module definitions to get a flavor for the type of information in a module.

The managed objects falling under system contain general information about the device being managed; all managed devices must support the system MIB objects. Table 9.2 defines the objects in the system group, as defined in [RFC 1213]. Table 9.3 defines the managed objects in the MIB module for the UDP protocol at a managed entity.

9.3.3
SNMP Protocol Operations and Transport Mappings

The Simple Network Management Protocol version 2 (SNMPv2) [RFC 3416] is used to convey MIB information among managing entities and agents executing on behalf of managing entities. The most common usage of SNMP is in a **request-response mode** in which an SNMPv2 managing entity sends a request to an SNMPv2 agent, who receives the request, performs some action, and sends a reply to the request. Typically, a request will be used to query (retrieve) or modify (set) MIB object values

Object Identifier	Name	Type	Description (from RFC 1213)
1.3.6.1.2.1.1.1	sysDescr	OCTET STRING	“Full name and version identification of the system’s hardware type, software operating-system, and networking software.”
1.3.6.1.2.1.1.2	sysObjectID	OBJECT IDENTIFIER	Vendor-assigned object ID that “provides an easy and unambiguous means for determining ‘what kind of box’ is being managed.”
1.3.6.1.2.1.1.3	sysUpTime	TimeTicks	“The time (in hundredths of a second) since the network management portion of the system was last re-initialized.”
1.3.6.1.2.1.1.4	sysContact	OCTET STRING	“The contact person for this managed node, together with information on how to contact this person.”
1.3.6.1.2.1.1.5	sysName	OCTET STRING	“An administratively assigned name for this managed node. By convention, this is the node’s fully qualified domain name.”
1.3.6.1.2.1.1.6	sysLocation	OCTET STRING	“The physical location of this node.”
1.3.6.1.2.1.1.7	sysServices	Integer32	A coded value that indicates the set of services available at this node: physical (for example, a repeater), data link/subnet (for example, bridge), Internet (for example, IP gateway), end-to-end (for example, host), applications.

Table 9.2 ♦
Managed objects in the MIB-2 system group

Object Identifier	Name	Type	Description (from RFC 4113)
1.3.6.1.2.1.7.1	udpInDatagrams	Counter32	“total number of UDP datagrams delivered to UDP users”
1.3.6.1.2.1.7.2	udpNoPorts	Counter32	“total number of received UDP datagrams for which there was no application at the destination port”
1.3.6.1.2.1.7.3	udpInErrors	Counter32	“number of received UDP datagrams that could not be delivered for reasons other than the lack of an application at the destination port”
1.3.6.1.2.1.7.4	udpOutDatagrams	Counter32	“total number of UDP datagrams sent from this entity”

Table 9.3 ♦ Selected managed objects in the MIB-2 UDP module

associated with a managed device. A second common usage of SNMP is for an agent to send an unsolicited message, known as a **trap message**, to a managing entity. Trap messages are used to notify a managing entity of an exceptional situation that has resulted in changes to MIB object values. We saw earlier in Section 9.1 that the network administrator might want to receive a trap message, for example, when an interface goes down, congestion reaches a predefined level on a link, or some other noteworthy event occurs. Note that there are a number of important trade-offs between polling (request-response interaction) and trapping; see the homework problems.

SNMPv2 defines seven types of messages, known generically as protocol data units—PDUs—as shown in Table 9.4 and described next. The format of the PDU is shown in Figure 9.4.

- The **GetRequest**, **GetNextRequest**, and **GetBulkRequest** PDUs are all sent from a managing entity to an agent to request the value of one or more MIB objects at the agent’s managed device. The object identifiers of the MIB objects whose values are being requested are specified in the variable binding portion of the PDU. **GetRequest**, **GetNextRequest**, and **GetBulkRequest** differ in the granularity of their data requests. **GetRequest** can request an arbitrary set of MIB values; multiple **GetNextRequests** can be used to sequence through a list or table of MIB objects; **GetBulkRequest** allows a large block of data to be returned, avoiding the overhead incurred if multiple **GetRequest** or **GetNextRequest** messages were to be sent. In all three cases, the agent responds with a **Response** PDU containing the object identifiers and their associated values.
- The **SetRequest** PDU is used by a managing entity to set the value of one or more MIB objects in a managed device. An agent replies with a **Response** PDU with the “noError” error status to confirm that the value has indeed been set.

SNMPv2 PDU Type	Sender-receiver	Description
GetRequest	manager-to-agent	get value of one or more MIB object instances
GetNextRequest	manager-to-agent	get value of next MIB object instance in list or table
GetBulkRequest	manager-to-agent	get values in large block of data, for example, values in a large table
InformRequest	manager-to-manager	inform remote managing entity of MIB values remote to its access
SetRequest	manager-to-agent	set value of one or more MIB object instances
Response	agent-to-manager or manager-to-manager	generated in response to
		GetRequest,
		GetNextRequest,
		GetBulkRequest,
		SetRequest PDU, or
		InformRequest
SNMPv2-Trap	agent-to-manager	inform manager of an exceptional event

Table 9.4 ♦ SNMPv2 PDU types

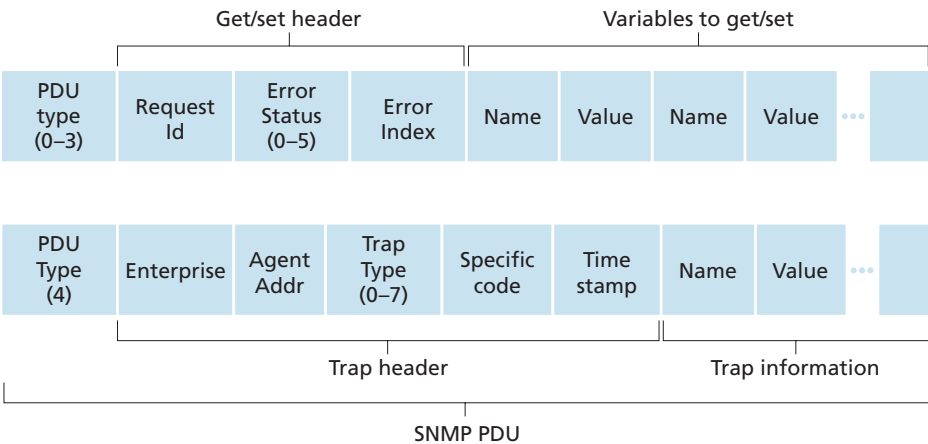


Figure 9.4 ♦ SNMP PDU format

- The **InformRequest** PDU is used by a managing entity to notify another managing entity of MIB information that is remote to the receiving entity. The receiving entity replies with a **Response** PDU with the “noError” error status to acknowledge receipt of the **InformRequest** PDU.
- The final type of SNMPv2 PDU is the trap message. Trap messages are generated asynchronously; that is, they are *not* generated in response to a received request but rather in response to an event for which the managing entity requires notification. RFC 3418 defines well-known trap types that include a cold or warm start by a device, a link going up or down, the loss of a neighbor, or an authentication failure event. A received trap request has no required response from a managing entity.

Given the request-response nature of SNMPv2, it is worth noting here that although SNMP PDUs can be carried via many different transport protocols, the SNMP PDU is typically carried in the payload of a UDP datagram. Indeed, RFC 3417 states that UDP is “the preferred transport mapping.” Since UDP is an unreliable transport protocol, there is no guarantee that a request, or its response, will be received at the intended destination. The request ID field of the PDU is used by the managing entity to number its requests to an agent; an agent’s response takes its request ID from that of the received request. Thus, the request ID field can be used by the managing entity to detect lost requests or replies. It is up to the managing entity to decide whether to retransmit a request if no corresponding response is received after a given amount of time. In particular, the SNMP standard does not mandate any particular procedure for retransmission, or even if retransmission is to be done in the first place. It only requires that the managing entity “needs to act responsibly in respect to the frequency and duration of retransmissions.” This, of course, leads one to wonder how a “responsible” protocol should act!

9.3.4 Security and Administration

The designers of SNMPv3 have said that “SNMPv3 can be thought of as SNMPv2 with additional security and administration capabilities” [RFC 3410]. Certainly, there are changes in SNMPv3 over SNMPv2, but nowhere are those changes more evident than in the area of administration and security. The central role of security in SNMPv3 was particularly important, since the lack of adequate security resulted in SNMP being used primarily for monitoring rather than control (for example, **SetRequest** is rarely used in SNMPv1).

As SNMP has matured through three versions, its functionality has grown but so too, alas, has the number of SNMP-related standards documents. This is evidenced by the fact that there is even now an RFC [RFC 3411] that “describes an architecture for describing SNMP Management Frameworks”! While the notion of an “architecture” for “describing a framework” might be a bit much to wrap one’s mind around, the goal of RFC 3411 is an admirable one—to introduce a common language for describing the functionality and actions taken by an SNMPv3 agent or

managing entity. The architecture of an SNMPv3 entity is straightforward, and a tour through the architecture will serve to solidify our understanding of SNMP.

So-called **SNMP applications** consist of a command generator, notification receiver, and proxy forwarder (all of which are typically found in a managing entity); a command responder and notification originator (both of which are typically found in an agent); and the possibility of other applications. The command generator generates the `GetRequest`, `GetNextRequest`, `GetBulkRequest`, and `SetRequest` PDUs that we examined in Section 9.3.3 and handles the received responses to these PDUs. The command responder executes in an agent and receives, processes, and replies (using the `Response` message) to received `GetRequest`, `GetNextRequest`, `GetBulkRequest`, and `SetRequest` PDUs. The notification originator application in an agent generates `Trap` PDUs; these PDUs are eventually received and processed in a notification receiver application at a managing entity. The proxy forwarder application forwards request, notification, and response PDUs.

A PDU sent by an SNMP application next passes through the SNMP “engine” before it is sent via the appropriate transport protocol. Figure 9.5 shows how a PDU generated by the command generator application first enters the dispatch module,

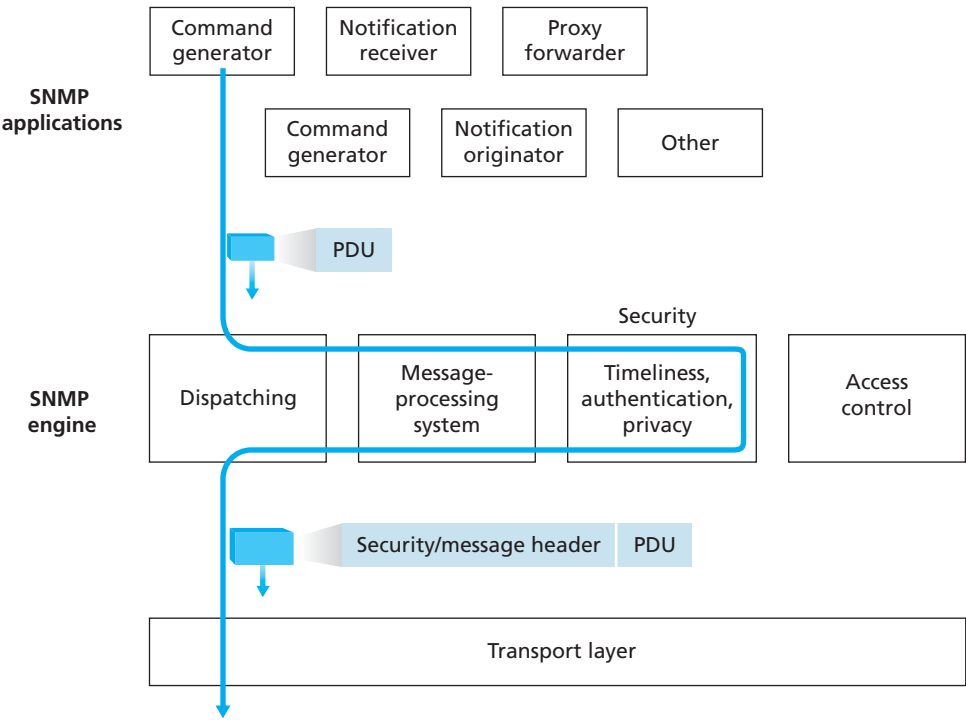


Figure 9.5 ♦ SNMPv3 engine and applications

where the SNMP version is determined. The PDU is then processed in the message-processing system, where the PDU is wrapped in a message header containing the SNMP version number, a message ID, and message size information. If encryption or authentication is needed, the appropriate header fields for this information are included as well; see [RFC 3411] for details. Finally, the SNMP message (the application-generated PDU plus the message header information) is passed to the appropriate transport protocol. The preferred transport protocol for carrying SNMP messages is UDP (that is, SNMP messages are carried as the payload in a UDP datagram), and the preferred port number for the SNMP is port 161. Port 162 is used for trap messages.

We have seen above that SNMP messages are used not just to monitor, but also to control (for example, through the `SetRequest` command) network elements. Clearly, an intruder that could intercept SNMP messages and/or generate its own SNMP packets into the management infrastructure could wreak havoc in the network. Thus, it is crucial that SNMP messages be transmitted securely. Surprisingly, it is only in the most recent version of SNMP that security has received the attention that it deserves. SNMPv3 security is known as **user-based security** [RFC 3414] in that there is the traditional concept of a user, identified by a username, with which security information such as a password, key value, or access privileges are associated. SNMPv3 provides for encryption, authentication, protection against playback attacks (see Section 8.3), and access control.

- *Encryption.* SNMP PDUs can be encrypted using the Data Encryption Standard (DES) in Cipher Block Chaining (CBC) mode. Note that since DES is a shared-key system, the secret key of the user encrypting data must be known by the receiving entity that must decrypt the data.
- *Authentication.* SNMP uses the Message Authentication Code (MAC) technique that we studied in Section 8.3.1 to provide both authentication and protection against tampering [RFC 4301]. Recall that a MAC requires the sender and receiver both to know a common secret key.
- *Protection against playback.* Recall from our discussion in Chapter 8 that nonces can be used to guard against playback attacks. SNMPv3 adopts a related approach. In order to ensure that a received message is not a replay of some earlier message, the receiver requires that the sender include a value in each message that is based on a counter in the *receiver*. This counter, which functions as a nonce, reflects the amount of time since the last reboot of the receiver's network management software and the total number of reboots since the receiver's network management software was last configured. As long as the counter in a received message is within some margin of error of the receiver's actual value, the message is accepted as a nonreplay message, at which point it may be authenticated and/or decrypted. See [RFC 3414] for details.
- *Access control.* SNMPv3 provides a view-based access control [RFC 3415] that controls which network management information can be queried and/or set by which users. An SNMP entity retains information about access rights and

policies in a Local Configuration Datastore (LCD). Portions of the LCD are themselves accessible as managed objects, defined in the View-Based Access Control Model Configuration MIB [RFC 3415], and thus can be managed and manipulated remotely via SNMP.

9.4 ASN.1

In this book, we have covered a number of interesting topics in computer networking. This section on ASN.1, however, may not make the top-ten list of interesting topics. Like vegetables, knowledge about ASN.1 and the broader issue of presentation services is something that is “good for you.” ASN.1 is an ISO-originated standard that is used in a number of Internet-related protocols, particularly in the area of network management. For example, we saw in Section 9.3 that MIB variables in SNMP were inextricably tied to ASN.1. So while the material on ASN.1 in this section may be rather dry, we hope the reader will take it on faith that the material *is* important.

In order to motivate our discussion here, consider the following thought experiment. Suppose one could reliably copy data from one computer’s memory directly into a remote computer’s memory. If one could do this, would the communication problem be “solved?” The answer to the question depends on one’s definition of “the communication problem.” Certainly, a perfect memory-to-memory copy would exactly communicate the bits and bytes from one machine to another. But does such an exact copy of the bits and bytes mean that when software running on the receiving computer accesses this data, it will see the same values that were stored into the sending computer’s memory? The answer to this question is “not necessarily!” The crux of the problem is that different computer architectures, different operating systems, and different compilers have different conventions for storing and representing data. If data is to be communicated and stored among multiple computers (as it is in every communication network), this problem of data representation must clearly be solved.

As an example of this problem, consider the simple C code fragment below. How might this structure be laid out in memory?

```
struct {
    char code;
    int x;
} test;
test.x = 259;
test.code = 'a';
```

The left side of Figure 9.6 shows a possible layout of this data on one hypothetical architecture: there is a single byte of memory containing the character a,

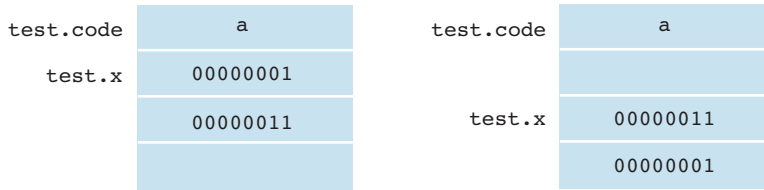


Figure 9.6 ♦ Two different data layouts on two different architectures

followed by a 16-bit word containing the integer value 259, stored with the most significant byte first. The layout in memory on another computer is shown in the right half of Figure 9.6. The character `a` is followed by the integer value stored with the least significant byte stored first and with the 16-bit integer aligned to start on a 16-bit word boundary. Certainly, if one were to perform a verbatim copy between these two computers' memories and use the same structure definition to access the stored values, one would see very different results on the two computers!

The fact that different architectures have different internal data formats is a real and pervasive problem. The particular problem of integer storage in different formats is so common that it has a name. “Big-endian” order for storing integers has the most significant bytes of the integer stored first (at the lowest storage address). “Little-endian” order stores the least significant bytes first. Sun SPARC and Motorola processors are big-endian, while Intel processors are little-endian. As an aside, the terms “big-endian” and “little-endian” come from the book, *Gulliver's Travels*, by Jonathan Swift, in which two groups of people dogmatically insist on doing a simple thing in two different ways (hopefully, the analogy to the computer architecture community is clear). One group in the land of Lilliput insists on breaking their eggs at the larger end (“the big-endians”), while the other insists on breaking them at the smaller end. The difference was the cause of great civil strife and rebellion.

Given that different computers store and represent data in different ways, how should networking protocols deal with this? For example, if an SNMP agent is about to send a Response message containing the integer count of the number of received UDP datagrams, how should it represent the integer value to be sent to the managing entity—in big-endian or little-endian order? One option would be for the agent to send the bytes of the integer in the same order in which they would be stored in the managing entity. Another option would be for the agent to send in its own storage order and have the receiving entity reorder the bytes, as needed. Either option would require the sender or receiver to learn the other's format for integer representation.

A third option is to have a machine-independent, OS-independent, language-independent method for describing integers and other data types (that is, a data-definition language) and rules that state the manner in which each of the data types is to be transmitted over the network. When data of a given type is received, it is received in a known format and can then be stored in whatever machine-specific format is required. Both the SMI that we studied in Section 9.3 and ASN.1 adopt this third option. In ISO parlance, these two standards describe a **presentation service**—the service of transmitting and translating information from one machine-specific format to another. Figure 9.7 illustrates a real-world presentation problem; neither receiver understands the essential idea being communicated—that the speaker likes something. As shown in Figure 9.8, a presentation service can solve this problem by translating the idea into a commonly understood (by the presentation service), person-independent language, sending that information to the receiver, and then translating into a language understood by the receiver.

Table 9.5 shows a few of the ASN.1-defined data types. Recall that we encountered the INTEGER, OCTET STRING, and OBJECT IDENTIFIER data types in our earlier study of the SMI. Since our goal here is (mercifully) not to provide a complete introduction to ASN.1, we refer the reader to the standards or to the printed and online book [Larmouth 1996] for a description of ASN.1 types and constructors, such as SEQUENCE and SET, that allow for the definition of structured data types.

In addition to providing a data definition language, ASN.1 also provides **Basic Encoding Rules (BER)** that specify how instances of objects that have been defined using the ASN.1 data definition language are to be sent over the network. The BER adopts a so-called **TLV (Type, Length, Value) approach** to encoding data for transmission. For each data item to be sent, the data type, the length of the data item,

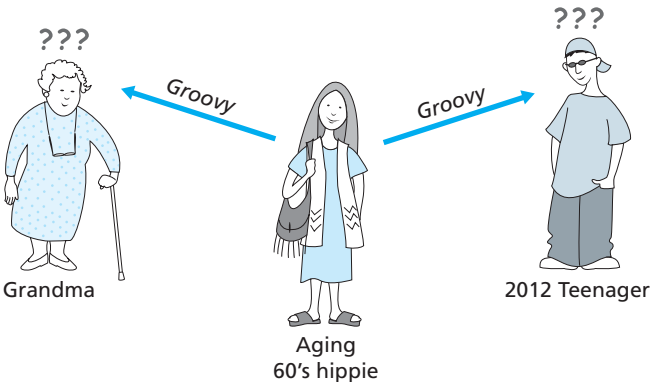


Figure 9.7 ♦ The presentation problem

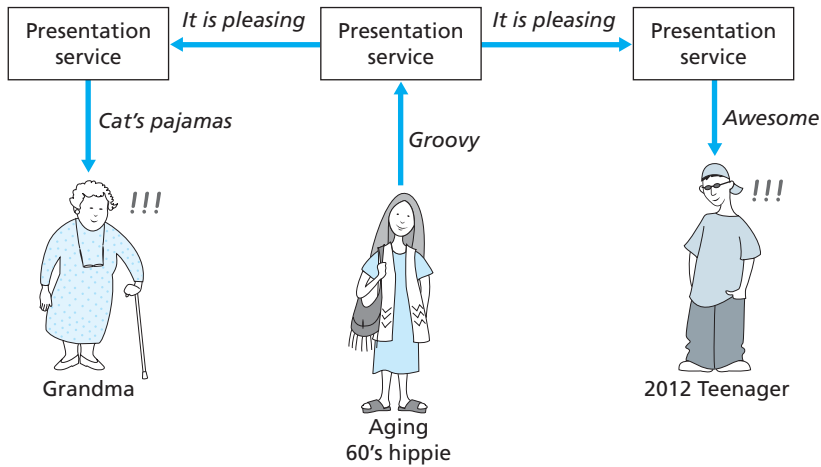


Figure 9.8 ♦ The presentation problem solved

and then the actual value of the data item are sent, in that order. With this simple convention, the received data is essentially self-identifying.

Figure 9.9 shows how the two data items in a simple example would be sent. In this example, the sender wants to send the character string “smith” followed by the value 259 decimal (which equals 00000001 00000011 in binary, or a byte value of 1 followed by a byte value of 3), assuming big-endian order. The first byte in the

Tag	Type	Description
1	BOOLEAN	value is “true” or “false”
2	INTEGER	can be arbitrarily large
3	BITSTRING	list of one or more bits
4	OCTET STRING	list of one or more bytes
5	NULL	no value
6	OBJECT IDENTIFIER	name, in the ASN.1 standard naming tree; see Section 9.2.2
9	REAL	floating point

Table 9.5 ♦ Selected ASN.1 data types

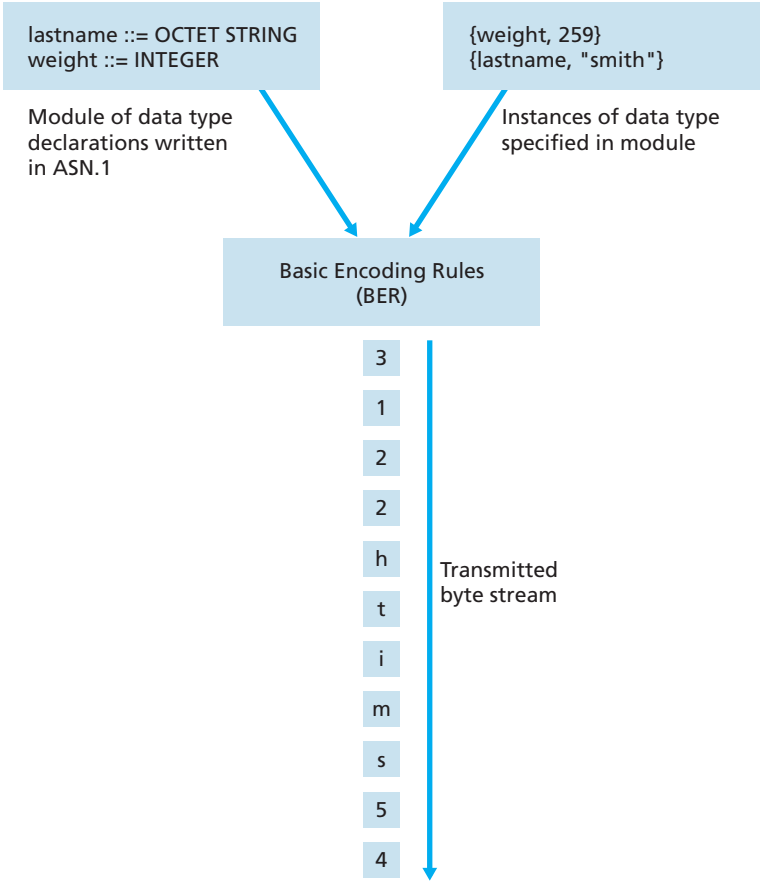


Figure 9.9 ♦ BER encoding example

transmitted stream has the value 4, indicating that the type of the following data item is an OCTET STRING; this is the “T” in the TLV encoding. The second byte in the stream contains the length of the OCTET STRING, in this case 5. The third byte in the transmitted stream begins the OCTET STRING of length 5; it contains the ASCII representation of the letter *s*. The T, L, and V values of the next data item are 2 (the INTEGER type tag value), 2 (that is, an integer of length 2 bytes), and the 2-byte big-endian representation of the value 259 decimal.

In our previous discussion, we have only touched on a small and simple subset of ASN.1. Resources for learning more about ASN.1 include the ASN.1 standards document [ISO X.680 2002], the online OSI-related book [Larmouth 2012], and the ASN.1-related Web sites, [OSS 2012] and [OID Repository 2012].

9.5 Conclusion

Our study of network management, and indeed of all of networking, is now complete!

In this final chapter on network management, we began by motivating the need for providing appropriate tools for the network administrator—the person whose job it is to keep the network “up and running”—for monitoring, testing, polling, configuring, analyzing, evaluating, and controlling the operation of the network. Our analogies with the management of complex systems such as power plants, airplanes, and human organization helped motivate this need. We saw that the architecture of network management systems revolves around five key components: (1) a network manager, (2) a set of managed remote (from the network manager) devices, (3) the Management Information Bases (MIBs) at these devices, containing data about the devices’ status and operation, (4) remote agents that report MIB information and take action under the control of the network manager, and (5) a protocol for communication between the network manager and the remote devices.

We then delved into the details of the Internet-Standard Management Framework, and the SNMP protocol in particular. We saw how SNMP instantiates the five key components of a network management architecture, and we spent considerable time examining MIB objects, the SMI—the data definition language for specifying MIBs, and the SNMP protocol itself. Noting that the SMI and ASN.1 are inextricably tied together, and that ASN.1 plays a key role in the presentation layer in the ISO/OSI seven-layer reference model, we then briefly examined ASN.1. Perhaps more important than the details of ASN.1 itself was the noted need to provide for translation between machine-specific data formats in a network. While some network architectures explicitly acknowledge the importance of this service by having a presentation layer, this layer is absent in the Internet protocol stack.

It is also worth noting that there are many topics in network management that we chose *not* to cover—topics such as fault identification and management, proactive anomaly detection, alarm correlation, and the larger issues of service management (for example, as opposed to network management). While important, these topics would form a text in their own right, and we refer the reader to the references noted in Section 9.1.



Homework Problems and Questions

Chapter 9 Review Questions

SECTION 9.1

- R1. Why would a network manager benefit from having network management tools? Describe five scenarios.
- R2. What are the five areas of network management defined by the ISO?

- R3. What is the difference between network management and service management?

SECTION 9.2

- R4. Define the following terms: managing entity, managed device, management agent, MIB, network management protocol.

SECTION 9.3

- R5. What is the role of the SMI in network management?
- R6. What is an important difference between a request-response message and a trap message in SNMP?
- R7. What are the seven message types used in SNMP?
- R8. What is meant by an “SNMP engine”?

SECTION 9.4

- R9. What is the purpose of the ASN.1 object identifier tree?
- R10. What is the role of ASN.1 in the ISO/OSI reference model’s presentation layer?
- R11. Does the Internet have a presentation layer? If not, how are concerns about differences in machine architectures—for example, the different representation of integers on different machines—addressed?
- R12. What is meant by TLV encoding?



Problems

- P1. Consider the two ways in which communication occurs between a managing entity and a managed device: request-response mode and trapping. What are the pros and cons of these two approaches, in terms of (1) overhead, (2) notification time when exceptional events occur, and (3) robustness with respect to lost messages between the managing entity and the device?
- P2. In Section 9.3 we saw that it was preferable to transport SNMP messages in unreliable UDP datagrams. Why do you think the designers of SNMP chose UDP rather than TCP as the transport protocol of choice for SNMP?
- P3. What is the ASN.1 object identifier for the ICMP protocol (see Figure 9.3)?
- P4. Suppose you worked for a US-based company that wanted to develop its own MIB for managing a product line. Where in the object identifier tree (Figure 9.3) would it be registered? (*Hint:* You’ll have to do some digging through RFCs or other documents to answer this question.)

- P5. Recall from Section 9.3.2 that a private company (enterprise) can create its own MIB variables under the private branch 1.3.6.4. Suppose that IBM wanted to create a MIB for its Web server software. What would be the next OID qualifier after 1.3.6.1.4? (In order to answer this question, you will need to consult [IANA 2009b]). Search the Web and see if you can find out whether such a MIB exists for an IBM server.
- P6. Why do you think the length precedes the value in a TLV encoding (rather than the length following the value)?
- P7. Consider Figure 9.9. What would be the BER encoding of {weight, 165} {lastname, "Michael"}?
- P8. Consider Figure 9.9. What would be the BER encoding of {weight, 145} {lastname, "Sridhar"}?

Jennifer Rexford

Jennifer Rexford is a Professor in the Computer Science department at Princeton University. Her research has the broad goal of making computer networks easier to design and manage, with particular emphasis on routing protocols. From 1996–2004, she was a member of the Network Management and Performance department at AT&T Labs–Research. While at AT&T, she designed techniques and tools for network measurement, traffic engineering, and router configuration that were deployed in AT&T’s backbone network. Jennifer is co-author of the book “Web Protocols and Practice: Networking Protocols, Caching, and Traffic Measurement,” published by Addison-Wesley in May 2001. She served as the chair of ACM SIGCOMM from 2003 to 2007. She received her BSE degree in electrical engineering from Princeton University in 1991, and her MSE and PhD degrees in electrical engineering and computer science from the University of Michigan in 1993 and 1996, respectively. In 2004, Jennifer was the winner of ACM’s Grace Murray Hopper Award for outstanding young computer professional and appeared on the MIT TR-100 list of top innovators under the age of 35.



Please describe one or two of the most exciting projects you have worked on during your career. What were the biggest challenges?

When I was a researcher at AT&T, a group of us designed a new way to manage routing in Internet Service Provider backbone networks. Traditionally, network operators configure each router individually, and these routers run distributed protocols to compute paths through the network. We believed that network management would be simpler and more flexible if network operators could exercise *direct* control over how routers forward traffic based on a *network-wide* view of the topology and traffic. The Routing Control Platform (RCP) we designed and built could compute the routes for all of AT&T’s backbone on a single commodity computer, and could control legacy routers without modification. To me, this project was exciting because we had a provocative idea, a working system, and ultimately a real deployment in an operational network.

What changes and innovations do you see happening in network management in the future?

Rather than simply “bolting on” network management on top of existing networks, researchers and practitioners alike are starting to design networks that are fundamentally easier to manage. Like our early work on the RCP, the main idea in so-called Software Defined Networking (SDN) is to run a controller that can install low-level packet-handling rules in the underlying switches using a standard protocol. This controller can run various

network-management applications, such as dynamic access control, seamless user mobility, traffic engineering, server load balancing, energy-efficient networking, and so on. I believe SDN is a great opportunity to get network management right, by rethinking the relationship between the network devices and the software that manages them.

Where do you see the future of networking and the Internet?

Networking is an exciting field because the applications and the underlying technologies change all the time. We are always reinventing ourselves! Who would have predicted even five or ten years ago the dominance of smart phones, allowing mobile users to access existing applications as well as new location-based services? The emergence of cloud computing is fundamentally changing the relationship between users and the applications they run, and networked sensors are enabling a wealth of new applications. The pace of innovation is truly inspiring.

The underlying network is a crucial component in all of these innovations. Yet, the network is notoriously “in the way”—limiting performance, compromising reliability, constraining applications, and complicating the deployment and management of services. We should strive to make the network of the future as invisible as the air we breathe, so it never stands in the way of new ideas and valuable services. To do this, we need to raise the level of abstraction above individual network devices and protocols (and their attendant acronyms!), so we can reason about the network as a whole.

What people inspired you professionally?

I’ve long been inspired by Sally Floyd at the International Computer Science Institute. Her research is always purposeful, focusing on the important challenges facing the Internet. She digs deeply into hard questions until she understands the problem and the space of solutions completely, and she devotes serious energy into “making things happen,” such as pushing her ideas into protocol standards and network equipment. Also, she gives back to the community, through professional service in numerous standards and research organizations and by creating tools (such as the widely used ns-2 and ns-3 simulators) that enable other researchers to succeed. She retired in 2009 but her influence on the field will be felt for years to come.

What are your recommendations for students who want careers in computer science and networking?

Networking is an inherently interdisciplinary field. Applying techniques from other disciplines to networking problems is a great way to move the field forward. We’ve seen tremendous

breakthroughs in networking come from such diverse areas as queuing theory, game theory, control theory, distributed systems, network optimization, programming languages, machine learning, algorithms, data structures, and so on. I think that becoming conversant in a related field, or collaborating closely with experts in those fields, is a wonderful way to put networking on a stronger foundation, so we can learn how to build networks that are worthy of society's trust. Beyond the theoretical disciplines, networking is exciting because we create real artifacts that real people use. Mastering how to design and build systems—by gaining experience in operating systems, computer architecture, and so on—is another fantastic way to amplify your knowledge of networking to help change the world.